

Your application can handle thousands of writes... but can it handle millions of reads? Companies like Amazon, Netflix, and Uber solve this problem using a powerful architecture pattern called CQRS.



What is CQRS?



- CQRS = Command Query Responsibility Segregation
- Separates Read operations from Write operations
- Commands change data
- Queries retrieve data
- Different models for different responsibilities
- Designed for high-scale systems



Why CQRS?



- Most applications have more reads than writes
- Read and write workloads have different requirements
- Reduces database bottlenecks
- Improves scalability and performance
- Makes complex systems easier to manage



Commands vs Queries



Commands (Write Side):

- Create data
- Update data
- Delete data
- Change application state
- Usually require validation and business rules

Queries (Read Side):

- Fetch data
- Search data
- Generate reports
- Never modify data
- Optimized for fast data retrieval



CQRS Architecture Overview



- **Write Side**

- Receives Commands

- Validates business rules

- Updates Write Database

- **Read Side**

- Receives Queries

- Reads from Read Database

- Optimized for high-speed responses

- **Event Flow**

- Commands generate Events

- Events update Read Models

- Queries use updated Read Models



Database Strategy in CQRS



- **Logical CQRS**
- Same database
- Separate command and query code
- Easier to implement
- Good for learning and small projects
- **True CQRS**
- Separate Read and Write Databases
- Write DB: PostgreSQL / MySQL
- Read DB: Redis / Elasticsearch / MongoDB
- Maximum scalability and performance



Benefits of CQRS



- Better performance
- Independent scaling
- Faster read operations
- Cleaner architecture
- Easier maintenance
- Supports Event-Driven Systems
- Works well with Kafka and Microservices



Challenges of CQRS



- Increased complexity
- More components to manage
- Additional infrastructure
- Data synchronization challenges
- Eventual consistency considerations



What is Eventual Consistency?



Real-World Examples



CQRS + Event-Driven Architecture

